

## Laborator 2

### DPAC

As. drd. Dan Curăvale

## Lab 2: de la pachete de date la alerta

- In Lab 1 am analizat cateva tipuri de trafic in wireshark.
- Am ajuns la cateva concluzii despre indicatorii unui atac.
- Intrebare: Care ar fi niste trasaturi relevante pentru a diferentia tipurile de trafic?
- In Lab 2 trecem de la trafic brut la semnale utile pentru detectie  $\rightsquigarrow$  extragere de trasaturi.
- Logica laboratorului:
  - observam traficul din PCAP in python
  - extragem trasaturi la nivel de pachet si flow
  - transformam pattern-urile in reguli Snort
- Scop practic: sa putem explica *de ce* o alerta are sens, nu doar sa o vedem.

# De ce nu ajunge un pachet izolat?

- Un pachet spune ce s-a intamplat intr-un singur moment al comunicarii.
- Multe atacuri nu se disting printr-un pachet special, ci prin repetitie, ritm si dezechilibru.
- Port scan: multe incercari scurte catre porturi diferite, fara conexiuni complete.
- SYN flood: multe initieri TCP fara handshake complet si fara raspunsuri normale.
- De aceea trebuie sa trecem de la evenimente izolate la comportamentul unei conversatii.

| Nivel  | Ce reprezinta                        | La ce ne ajuta                                                 |
|--------|--------------------------------------|----------------------------------------------------------------|
| Pachet | O unitate individuala de trafic      | Vedem IP-uri, porturi, flags, TTL, payload, stare locala       |
| Flow   | O conversatie intre doua endpointuri | Vedem durata, volum, directie, handshake, ritm si dezechilibru |

- Un flow este identificat prin 5 caracteristice extrase din antetele pachetelor: IP sursa, IP destinatie, port sursa, port destinatie, protocol.
- Dupa ce extragem cele 5 caracteristici pentru fiecare pachet, sortam/grupam dupa aceste chei si obtinem toate pachetele din acelasi flow.

## Volum si ritm

- packet\_count
- total\_bytes
- avg\_packet\_size
- packets\_per\_second
- bytes\_per\_second

## Directie

- fwd\_packets, bwd\_packets
- fwd\_bytes, bwd\_bytes

## Handshake si terminare

- syn\_count, ack\_count
- fin\_count, rst\_count

# De ce avem nevoie de ambele?

| Nivel pachet                       | Nivel flow                                                           |
|------------------------------------|----------------------------------------------------------------------|
| Ce s-a intamplat acum?             | Cum s-a purtat conversatia?                                          |
| Un SYN catre portul 80             | Zeci de SYN catre porturi diferite                                   |
| Un POST sau un password in payload | Raport dezechilibrat intre forward si backward                       |
| Un ACK, un RST, un TTL             | $\text{syn\_count} > \text{ack\_count}$ , durata mica, volum anormal |

- O detectie eficienta imbina indiciile locale si contextul comportamental.
- Fara flow-uri vedem doar fragmente; fara analiza pe pachete nu mai putem observa detalii tehnice (flag-uri TCP, TTL, payload) care explica alerta.
- Flow  $\rightsquigarrow$  spune daca traficul e suspect.
- Pachet  $\rightsquigarrow$  spune de ce e suspect.

# Cum decurge partea practica in cod

- ➊ `load_pcap`: citim captura si vedem cate pachete avem.
- ➋ `explore_packets`: inspectam cateva exemple si protocoalele dominante.
- ➌ `extract_packet_features`: extragem campurile relevante per pachet.
- ➍ `packets_to_dataframe`: obtinem o vedere tabelara pentru analiza.
- ➎ `aggregate_flows`: grupam pachetele in conversatii si calculam statistici.
- ➏ Detectie simpla: cautam scaneri si flow-uri cu rata mare sau dezechilibru clar.
- ➐ Exportul exista doar ca punte spre laboratoarele urmatoare, nu ca scop in sine.

- **1) Ce face sistemul (reactie)**

- **IDS** (Intrusion Detection System): detecteaza si alerteaza.
- **IPS** (Intrusion Prevention System): detecteaza, alerteaza si poate bloca automat.

- **2) Unde observa (pozitionare)**

- **NIDS** (Network IDS): analizeaza traficul din retea, la nivel de pachet si flow.
- **HIDS** (Host IDS): analizeaza evenimente locale pe host (procese, loguri, fisiere, apeluri de sistem).

- **3) Cum detecteaza (metoda)**

- **Signature-based**: cauta semnaturi cunoscute; precizie buna, dar acoperire slaba pentru atacuri noi.
- **Anomaly-based**: cauta abateri fata de normal; utila pentru necunoscute, dar cu risc mai mare de false positives.



`ACTION PROTOCOL SRC_IP SRC_PORT -> DST_IP DST_PORT (OPTIONS)`

- O regula transforma un pattern observat in trafic intr-o conditie explicita de detectie.
- `msg` spune ce alerta vrem sa vedem.
- `sid` este identificatorul unic al regulii.
- `content` cauta text in payload.
- `flags:S` inseamna pachet TCP cu flag-ul SYN setat.
- Exemplu de legatura cu teoria: un pattern de scanare devine o regula despre SYN si porturi.

**De ce nu este suficient sa analizam doar pachete individuale atunci cand cautam atacuri?**

- A. Pentru ca Wireshark nu poate afisa toate campurile relevante
- B. Pentru ca multe atacuri se vad mai clar ca pattern de conversatie, nu intr-un singur pachet
- C. Pentru ca Snort functioneaza doar pe flow-uri
- D. Pentru ca pachetele TCP nu contin adrese IP

**Cum este identificat un flow in laboratorul 2?**

- A. Doar prin IP sursa si IP destinatie
- B. Prin 5 caracteristici: IP sursa, IP destinatie, port sursa, port destinatie, protocol
- C. Prin timestamp si dimensiunea pachetului
- D. Doar prin portul destinatie si protocol

Care dintre urmatoarele este un feature la nivel de flow, nu la nivel de pachet?

- A. ip\_ttl
- B. tcp\_flags
- C. packet\_count
- D. src\_port

**Ce comportament indica cel mai clar un port scan in datele laboratorului?**

- A. Un singur flow cu multe raspunsuri ACK
- B. Multe pachete UDP catre acelasi port
- C. Un IP sursa care trimite SYN catre multe porturi destinatie diferite
- D. Un pachet HTTP care contine textul password

**Care este limita principala a unui IDS signature-based precum Snort?**

- A. Nu poate analiza fisiere PCAP
- B. Nu poate detecta deloc trafic HTTP
- C. Nu poate detecta bine atacuri noi daca nu exista deja regula corespunzatoare
- D. Nu poate folosi porturi in reguli